

Universidad de Ingeniería y Tecnología — UTEC

Proyecto de Base de Datos I

Hito 2: Implementación en PostgreSQL, Optimización y Experimentación
con Índices

Curso: Base de Datos I

Profesor: Wilder Nina Choquehuayta

Tema: Plataforma de catálogo, lectura, reseñas y descubrimiento de materiales literarios

Integrantes:

Sergio Felipe Delgado Amado — 202310227

Antonio Jorge Guerrero Cedrón — 202510717

Luciana Mylene Melgarejo Quispe — 202420055

Repositorio: github.com/SefedeamU/Proyecto-BD-Hito-2

Índice

Sobre este informe	3
1. Implementación de la base de datos	3
1.1. Creación de tablas en PostgreSQL	3
1.1.1. Triggers	4
1.1.2. Vistas de usuario	5
1.2. Carga de datos	5
1.3. Los cuatro escenarios de volumen	5
1.4. Ajustes al modelo del Hito 1	6
2. Optimización y Experimentación	7
2.1. Consultas del experimento	7
2.1.1. Criterios de selección	7
2.1.2. Consultas SQL	7
2.2. Metodología	8
2.3. Índices del experimento	8
2.4. Plataforma de pruebas	9
2.5. Medición de tiempos	9
2.5.1. Sin índices	9
2.5.2. Con índices	9
2.6. Resultados	9
2.6.1. Q1 — Materiales de un autor	9
2.6.2. Q2 — Materiales de un género	9
2.6.3. Q3 — Materiales más populares	10
2.6.4. Q4 — Historial de lectura de un usuario	10
2.6.5. Q5 — Consulta por rango	10
2.7. El costo de los índices	10
2.8. Escalamiento por encima del millón (punto extra)	10
2.9. Análisis y discusión	10
3. Conclusiones	10
4. Anexos	11
4.1. Estructura del repositorio	11
4.2. Video de la experimentación	11
4.3. Planes de ejecución (Q5, base 1M)	11

Sobre este informe

Este documento corresponde al Hito 2 del proyecto. El Hito 1 (requisitos, modelo E-R y modelo relacional) se entregó por separado y no se reproduce aquí. El esquema implementado parte directamente de ese modelo; los ajustes que fue necesario hacer al pasar al código se explican en la Sección 1.4 y están todos documentados en `docs/cambios_respecto_hito1.md` dentro del repositorio.

El dominio es una plataforma de catálogo literario. La entidad central es **Material**, que representa libros, ensayos, revistas, poemas y audiolibros, y se relaciona con autores, géneros, subgéneros, editoriales, premios, ilustraciones, curiosidades, reseñas, likes e historial de lectura.

1. Implementación de la base de datos

1.1. Creación de tablas en PostgreSQL

El esquema se implementó en PostgreSQL a partir del modelo relacional del Hito 1. El script `sql/01_tables.sql` crea las 25 tablas en el orden correcto de dependencias. Para la traducción de tipos se siguió el diccionario de datos del Hito 1:

Tipo en el modelo relacional	Tipo en PostgreSQL
Bigint	BIGINT
Int	INTEGER
Varchar(n)	VARCHAR(n)
Date	DATE
Double Precision	NUMERIC(3,1) (ver Sección 1.4, cambio G)

Las restricciones de integridad del Hito 1 se implementaron de tres formas según su naturaleza:

- **Restricciones de columna o tabla** (NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK): para reglas que dependen solo de una fila o de una tabla. Por ejemplo, `CHECK (CHAR_LENGTH(Password) = 12)` y `CHECK (Edad >= 12)` en `Usuario`; `CHECK (Relevancia BETWEEN 1 AND 5)` en `Premio`; `CHECK (... BETWEEN 1 AND 10)` para los cuatro indicadores de contenido en `AgeRate`.
- **Columna discriminadora**: la herencia total y disjunta de `Material` se garantiza con la columna `Tipo VARCHAR(10) NOT NULL` y un `CHECK` que solo acepta los cinco valores válidos (ver cambio A).
- **Triggers**: para reglas que dependen de dos tablas a la vez y no se pueden expresar con un `CHECK` simple (ver Sección 1.1.1).

Las definiciones de `Material` y `Usuario`, que concentran la mayoría de las restricciones, son las siguientes:

```
CREATE TABLE Material (  
  Id          BIGINT          NOT NULL ,  
  Numero_Paginas  INTEGER      NOT NULL ,
```

```

Anio_Publicacion BIGINT NOT NULL,
Eslogan          VARCHAR(100),
Alias            VARCHAR(50),
Pais             VARCHAR(15) NOT NULL,
Idioma           VARCHAR(15) NOT NULL,
Tipo             VARCHAR(10) NOT NULL, -- discriminador de
          subtipo [cambio A]
AgeRate_Code     BIGINT NOT NULL,
Editorial_Id     BIGINT NOT NULL,
CONSTRAINT pk_material PRIMARY KEY (Id),
CONSTRAINT ck_material_tipo CHECK (Tipo IN ('Libro','Ensayo','
Revista','Poema','AudioBook')),
CONSTRAINT fk_material_agerate FOREIGN KEY (AgeRate_Code)
REFERENCES AgeRate (Code),
CONSTRAINT fk_material_editorial FOREIGN KEY (Editorial_Id)
REFERENCES Editorial (Id)
);

CREATE TABLE Usuario (
Username VARCHAR(50) NOT NULL,
Email    VARCHAR(100) NOT NULL,
Password VARCHAR(12) NOT NULL,
Nombre   VARCHAR(20) NOT NULL,
Apellido VARCHAR(20) NOT NULL,
Edad     INTEGER NOT NULL,
Telefono BIGINT NOT NULL,
Ciudad   VARCHAR(20) NOT NULL,
Rol      VARCHAR(20) NOT NULL,
CONSTRAINT pk_usuario PRIMARY KEY (Username, Email),
CONSTRAINT uq_usuario_username UNIQUE (Username),
CONSTRAINT uq_usuario_email UNIQUE (Email),
CONSTRAINT ck_usuario_password CHECK (CHAR_LENGTH>Password) = 12),
CONSTRAINT ck_usuario_edad CHECK (Edad >= 12)
);

```

El esquema completo incluye las tablas maestras (AgeRate, Editorial, Genero, Autor, Premio, Ilustracion, Curiosidad, Material, Usuario), las entidades débiles (Warning, SubGenero), los cinco subtipos de Material y las tablas de relación (Escribe, Pertenece, PerteneceSubGenero, Popularizo, Contiene, Tiene, Ganar, Leer, Likes, Resena).

1.1.1. Triggers

El script `sql/02_triggers.sql` implementa cuatro reglas del Hito 1 que dependen de más de una tabla:

- Año de publicación vs. fundación editorial.** Anio_Publicacion de un material debe ser mayor o igual al año de Fundacion de su editorial. Se valida con un trigger BEFORE INSERT OR UPDATE sobre Material.
- Fecha de reseña vs. publicación.** El año de la fecha de una reseña debe ser mayor o igual al Anio_Publicacion del material reseñado. También BEFORE INSERT OR UPDATE, esta vez sobre Resena.
- Todo material tiene al menos un autor.** Participación total del lado Material en Escribe. Se implementa como CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY

DEFERRED: la validación ocurre al COMMIT, no fila por fila. Esto es necesario porque la inserción de **Material** y la de **Escribe** no se pueden hacer en el mismo instante; el trigger diferido da margen hasta que se cierre la transacción.

- (d) **Todo género tiene al menos un autor representante.** Participación total en Popularizo. Mismo mecanismo diferido que el punto anterior.

1.1.2. Vistas de usuario

El script `sql/03_views.sql` define cinco vistas:

- **vista_material_basico**: datos básicos de cada material con su editorial y clasificación de edad.
- **vista_material_autores**: materiales con sus autores (relación **Escribe**).
- **vista_material_subgenero**: materiales con género y subgénero; requiere la tabla **PerteneceSubGenero** añadida en el Hito 2 (cambio I).
- **vista_popularidad_material**: cantidad de likes, lecturas y reseñas por material, junto con el promedio de puntaje.
- **vista_historial_usuario**: historial de lectura por usuario con las fechas.

1.2. Carga de datos

La carga se realiza con un generador de datos sintéticos propio, en la carpeta **faker/**. Está escrito en Python usando las librerías **Faker** y **psycpg**. El generador aplica el esquema completo antes de poblar, genera primero las tablas maestras y luego las de interacción para respetar las llaves foráneas, y envuelve la carga de **Material+Escribe** y **Género+Popularizo** en transacciones para que los triggers diferidos puedan validar al COMMIT. Usa COPY en lugar de INSERT fila por fila para que la carga sea eficiente, y una semilla fija (SEED=42) para que los datos sean reproducibles.

1.3. Los cuatro escenarios de volumen

Para el experimento se trabajó con cuatro bases de datos, una por escenario:

Base	Usuarios	Materiales	Likes	Lecturas	Reseñas
bd_literaria_1k	200	300	1 000	1 000	1 000
bd_literaria_10k	1 000	2 000	10 000	10 000	10 000
bd_literaria_100k	10 000	20 000	100 000	100 000	100 000
bd_literaria_1m	100 000	100 000	1 000 000	1 000 000	1 000 000

La base de 1M tiene en total aproximadamente 3.86 millones de filas sumando todas las tablas. Los cuatro escenarios se entregan como dumps restaurables en **dumps/** y pueden regenerarse con el faker usando la misma semilla.

1.4. Ajustes al modelo del Hito 1

Al escribir el código se identificaron once puntos donde el modelo relacional dejaba reglas del dominio sin mecanismo de cumplimiento, o donde había decisiones de diseño que convenía precisar antes de poblar las bases. Todos los cambios están marcados como comentarios [Cambio X] en los scripts y explicados con detalle en `docs/cambios_respecto_hito1.md`.

ID	Tipo	Descripción
A	Integridad	Columna Tipo en Material para garantizar herencia total y disjunta.
B	Regla de negocio	Trigger diferido: un material sin autores no puede quedar en la base.
C	Regla de negocio	Trigger diferido: un género sin al menos un autor representante no puede quedar en la base.
D	Modelo	Fecha entra a la PK de Leer para poder registrar relecturas.
E	Integridad	UNIQUE(Material_Id, Usuario_Username, Usuario_Email) en Resena : un usuario no puede reseñar el mismo material dos veces.
F	Integridad	CHECK (Likes >= 0) en Resena : el contador de likes no puede ser negativo.
G	Tipo de dato	Resena.Puntaje cambia de DOUBLE PRECISION a NUMERIC(3,1) : el Hito 1 especificaba un decimal exacto, y DOUBLE PRECISION no lo garantiza.
H	Integridad ref.	ON UPDATE CASCADE en las FK que apuntan a Usuario : si cambia el username o el email, las tablas dependientes se actualizan solas.
I	Modelo	Nueva tabla PerteneceSubGenero : el Hito 1 pedía poder buscar materiales por subgénero, pero el modelo relacional dejaba a SubGenero sin conexión con Material .
J	Experimento	La consulta de materiales populares se reescribió para evitar un producto cartesiano que distorsionaba los tiempos.
K	Experimento	Se quitaron los índices redundantes con la primera columna de alguna PK (no aportan mejora y ensucian el experimento).

Vale la pena detenerse en los cambios J y K porque afectan directamente los resultados del experimento. El cambio J corrige la consulta de popularidad: la versión original unía las tres tablas de interacción con **LEFT JOIN** directos sobre **Material**, lo que generaba un producto cartesiano (filas de likes por filas de lecturas por filas de reseñas por cada material). La versión corregida agrega cada tabla por separado en subconsultas y luego las une, de modo que el tiempo que se mide refleja el costo real de una agregación total y no un error de formulación. El cambio K retira índices como el de **Material_Id** en

Escribe: como esa columna es la primera de la PK de **Escribe**, ya existe un índice de PK sobre ella y crear uno secundario encima no cambia nada; incluirlo en el experimento solo añadiría una fila de resultados con 0 % de mejora.

2. Optimización y Experimentación

2.1. Consultas del experimento

2.1.1. Criterios de selección

Se eligieron cinco consultas que cubren patrones de acceso distintos. El criterio principal fue que el conjunto en su totalidad mostrara los tres comportamientos posibles de un índice secundario: cuándo ayuda mucho, cuándo ayuda poco y cuándo no ayuda. Una de las consultas es por rango (**BETWEEN**), que es el caso más ilustrativo del funcionamiento de un índice B-tree.

ID	Qué devuelve	Tipo de acceso	Índice relevante
Q1	Materiales de un autor	Filtro selectivo por igualdad	<code>idx_escribe_autor</code>
Q2	Materiales de un género	Filtro poco selectivo por igualdad	<code>idx_pertenece_genero</code>
Q3	Los 20 materiales más populares	Agregación total sin filtro	(ninguno aplica)
Q4	Historial de lectura de un usuario	Igualdad + orden por fecha	<code>idx_leer_usuario</code>
Q5	Materiales en un rango de años	Rango (BETWEEN)	<code>idx_material_anio</code>

2.1.2. Consultas SQL

Las consultas están limpias en `quers/` y con **EXPLAIN** (**ANALYZE**, **BUFFERS**) en `sql/06_experiments.s`. Se reproducen aquí las dos más representativas del experimento.

Q5 — Materiales por rango de años.

```
SELECT m.Id, m.Alias, m.Anio_Publicacion, m.Idioma
FROM   Material m
WHERE  m.Anio_Publicacion BETWEEN 1900 AND 1905
ORDER BY m.Anio_Publicacion;
```

El rango elegido (primeros años del catálogo) es selectivo: recupera aproximadamente el 0.1% de las filas en la base de 1M. Eso permite que el planificador use el índice `idx_material_anio` de forma natural, sin forzarlo. En la Sección ?? se analiza qué pasa cuando el rango es más amplio.

Q4 — Historial de lectura de un usuario.

```
SELECT m.Id, m.Alias, le.Fecha
FROM Usuario u
JOIN Leer le ON le.Usuario_Username = u.Username
              AND le.Usuario_Email = u.Email
JOIN Material m ON m.Id = le.Material_Id
WHERE u.Username = :username
      AND u.Email = :email
ORDER BY le.Fecha DESC;
```

El índice `idx_leer_usuario` es compuesto: (`Usuario_Username`, `Usuario_Email`, `Fecha DESC`). Además de localizar las filas del usuario, entrega los resultados ya ordenados por fecha descendente, lo que evita el paso de `Sort`.

2.2. Metodología

- **Herramienta de medición:** `EXPLAIN (ANALYZE, BUFFERS)`, que ejecuta la consulta y reporta el tiempo real y el plan elegido por el planificador.
- **Número de corridas:** seis por consulta. La primera se descarta (estado frío) y se trabaja con la mediana de las cinco restantes para tener una medida estable en estado caliente.
- **Limpieza de caché entre corridas:** `DISCARD ALL` antes de cada corrida, para que el planificador vuelva a construir el plan desde cero y no reutilice planes cacheados de corridas anteriores.
- **Escenario sin índices:** se ejecuta `sql/05_drop_indexes.sql`, que elimina todos los índices secundarios. Los de `PRIMARY KEY` y `UNIQUE` no se pueden quitar porque son parte del esquema; por eso las consultas del experimento filtran por columnas que no son la primera de ninguna PK (`Autor_Id`, `Genero_Nombre`, `Usuario_Username/Email`, `Anio_Publicacion`) o son agregaciones totales donde ningún índice aplica (Q3).
- **Escenario con índices:** se ejecuta `sql/04_indexes.sql` y luego `ANALYZE` para que el planificador tenga estadísticas actualizadas.
- El protocolo se repite para las cuatro bases.

2.3. Índices del experimento

Nombre	Definición
<code>idx_escribe_autor</code>	<code>Escribe(Autor_Id)</code>
<code>idx_pertenece_genero</code>	<code>Pertenece(Genero_Nombre)</code>
<code>idx_resena_material</code>	<code>Resena(Material_Id)</code>
<code>idx_leer_usuario</code>	<code>Leer(Usuario_Username, Usuario_Email, Fecha DESC)</code>
<code>idx_material_anio</code>	<code>Material(Anio_Publicacion)</code>

2.4. Plataforma de pruebas

Las mediciones se realizaron sobre PostgreSQL 16 en Ubuntu 24.04, con la configuración por defecto del servidor, en las cuatro bases descritas en la Sección 1.3.

2.5. Medición de tiempos

2.5.1. Sin índices

Consulta	1K (ms)	10K (ms)	100K (ms)	1M (ms)
Q1 (autor)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q2 (género)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q3 (populares)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q4 (historial)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q5 (rango)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>

2.5.2. Con índices

Consulta	1K (ms)	10K (ms)	100K (ms)	1M (ms)
Q1 (autor)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q2 (género)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q3 (populares)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q4 (historial)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>
Q5 (rango)	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>	<i>pendiente</i>

2.6. Resultados

[Sección pendiente de completar una vez obtenidos los tiempos del servidor.]

2.6.1. Q1 — Materiales de un autor

[Pendiente. Incluir: tiempos, plan EXPLAIN sin y con índice, explicación de por qué `idx_escribe_autor` ayuda.]

2.6.2. Q2 — Materiales de un género

[Pendiente. Incluir: tiempos, plan EXPLAIN sin y con índice, explicación del impacto de la selectividad del filtro.]

2.6.3. Q3 — Materiales más populares

[Pendiente. Incluir: tiempos, plan EXPLAIN sin y con índice, explicación de por qué el índice no aporta en agregaciones totales.]

2.6.4. Q4 — Historial de lectura de un usuario

[Pendiente. Incluir: tiempos, plan EXPLAIN sin y con índice, explicación de `idx_leer_usuario` y cómo evita el paso de Sort.]

2.6.5. Q5 — Consulta por rango

[Pendiente. Incluir: tiempos, plan EXPLAIN sin y con índice, explicación del Bitmap Index Scan y la bondad del índice B-tree en predicados BETWEEN selectivos. También analizar qué ocurre con rangos amplios (el punto de cruce por selectividad).]

2.7. El costo de los índices

[Pendiente. El script `sql/07_index_overhead.sql` mide el costo en escrituras y en lecturas no selectivas. Ejecutarlo sobre `bd_literaria_1m` y completar con los resultados.]

2.8. Escalamiento por encima del millón (punto extra)

[Pendiente. Una vez obtenidos los tiempos de las cuatro bases, graficar el crecimiento del costo sin índices para ver si el patrón es lineal, y comparar con el comportamiento con índices. Discutir qué estrategias serían necesarias para volúmenes mayores al millón (vistas materializadas, particionamiento, réplicas de lectura).]

2.9. Análisis y discusión

[Pendiente. Una vez con los datos completos, redactar el análisis comparativo de los tres comportamientos observados: cuándo el índice ayuda mucho, cuándo poco y cuándo no ayuda. Incluir la aceleración (speedup) de cada consulta en la base de 1M como referencia principal.]

3. Conclusiones

- Se implementó en PostgreSQL el modelo relacional del Hito 1: 25 tablas con restricciones de integridad (PK, FK, UNIQUE, NOT NULL, CHECK), cuatro triggers para reglas multi-tabla (dos de ellos diferidos) y cinco vistas de usuario. Los once ajustes al modelo se documentaron de forma transparente.

- Se generaron cuatro escenarios de volumen reproducibles, hasta aproximadamente 3.86 millones de filas en el escenario de 1M, mediante un generador de datos sintéticos propio con semilla fija y carga por COPY.
- *[Pendiente: agregar las conclusiones del experimento una vez obtenidos los tiempos reales. Deben mencionar qué consultas se beneficiaron del índice y en qué medida, el comportamiento de la consulta por rango (Q5), y el costo de los índices en escrituras.]*

4. Anexos

4.1. Estructura del repositorio

Archivo o carpeta	Contenido
sql/01_tables.sql	Tablas con todas sus restricciones.
sql/02_triggers.sql	Funciones y triggers.
sql/03_views.sql	Vistas de usuario.
sql/04_indexes.sql	Índices del experimento.
sql/05_drop_indexes.sql	Quita los índices secundarios.
sql/06_experiments.sql	Consultas con EXPLAIN (ANALYZE, BUFFERS).
sql/07_index_overhead.sql	Demostración del costo de los índices.
queries/	Las cinco consultas limpias (Q1–Q5).
faker/	Generador de datos sintéticos.
dumps/	Dumps restaurables de las cuatro bases.
results/	CSV de resultados y gráficos (pendiente).
docs/	Este informe y el detalle de cambios.

4.2. Video de la experimentación

[Pendiente. El video debe mostrar la experimentación completa sobre la base de 1 millón de registros: medición sin índices, creación de índices, medición con índices y comparación de planes y tiempos. El guion detallado está en `video/README_guion.md` del repositorio.]

4.3. Planes de ejecución (Q5, base 1M)

[Pendiente. Capturar la salida de EXPLAIN (ANALYZE, BUFFERS) para la consulta Q5 sobre `bd_literaria_1m`, primero sin el índice `idx_material_anio` y luego con él. Pegar los resultados aquí.]